

LASPATED: A Library for the Analysis of Spatio-Temporal Discrete Data (User Manual)

V. Guigues
EMAp
FGV

A. J. Kleywegt
Industrial and
Systems Engineering
Georgia Institute
of Technology

G. Amorim
EMAp
FGV

A. Krauss
Department of
Informatics
PUC-RJ

V. H. Nascimento
EMAp
FGV

June 15, 2026

Abstract

This is the user manual of the LASPATED library. This library is available on GitHub (at <https://github.com/vguigues/LASPATED>) and provides a set of tools to analyze spatio-temporal data. A video tutorial for this library is available on YouTube. It is made of a Python package for time and space discretizations and a C++ package implementing the calibration of the probabilistic models for stochastic spatio-temporal data proposed in the companion paper Guigues et al. (2023).

1 Introduction

LASPATED is a library for the analysis of spatio-temporal data. It has a Python module for space and time discretization and a module for calibration of probabilistic models for such data. LASPATED takes as input a file of events with spatio-temporal information (a .csv file for example) or the corresponding Python DataFrame object. Each event has a location, a timestamp, and possibly a set of features.¹ LASPATED provides three different types of space discretization (in rectangles, hexagons, or customized) and several types of time discretization for such events. LASPATED also allows combining information from two different space discretizations. It computes the areas of intersection of all the subregions from the two discretizations. Given some attribute (for instance, the population) from the first discretization, LASPATED also computes the values of this attribute in the regions of the second discretization.

Time discretizations provide different partitions of time, for instance periodic with time windows of equal duration, periodic with time windows of different durations, or more general partitions, see Section 4.

The calibration module considers a discrete spatio-temporal time series, possibly built by the discretization module. The underlying model assumes the number of arrivals of type c , for zone i , and time window t , is Poisson with some intensity $\lambda_{c,i,t}$. Two strategies are implemented for the calibration of these intensities: maximizing a regularized Likelihood with penalizing weights obtained by cross validation and a model with covariates. The corresponding optimization problems are solved using projected gradient with line search along the feasible direction.

The outline of this manual is the following. Sections 3 to 7 describe the discretization tools. In Section 3, we explain how to load data. Time and space discretization functions are respectively discussed in Sections 4 and 5. Additional discretization features are provided in Section 6 while Section 7 explains how to access discretized data and how to write such data in files. Section 8 discusses the C++ functions for the calibration of the probabilistic models. Our code is tested using GCC 11.4 and Python 3.10. The projection with covariates was solved using Gurobi 11.1.0.

¹We use the terms events and arrivals with the same meaning in this manual.

2 Python installation guide

The LASPATED Python library is available at PyPI and can be installed with pip. Simply run:

```
$ pip install laspated
```

Throughout this manual, the \$ sign represents the command line interface chosen by the user, whether it is Windows or Linux. The Docker container (see section 11) uses Linux Ubuntu. After running pip, the Python module and all its dependencies will be automatically installed.

3 Initializing a data aggregator and adding events

LASPATED library is imported with command **import laspated**. The library uses a *DataAggregator* object of spatial and temporal information.

There are multiple options when using latitude-longitude numbers to represent locations. When initializing the data aggregator object, we should specify the CRS (Coordinates Reference System) used in the data. It is important to note that our data can be passed to different CRS, but LASPATED will use the specified CRS when aggregating information, for instance we can choose `app = spated.DataAggregator(crs="epsg:4326")`, see Example 1 below.

The observations we are interested in are spatio-temporal observations which have a date and time information, a location information, and possibly additional features. As an example, consider the following 19 first lines of a CSV file reporting emergency calls to Rio de Janeiro emergency health service (a given line of this file available on the GitHub page of the project corresponds to a call and the information we have on the calls are the time of the call, the location given by latitude and longitude, and the priority of the call coded by an integer between 0 and 3):

```
date_time;priority;long;lat
01/01/2016 02:09;3;-43.33366;-22.86939
01/01/2016 02:13;0;-43.67589;-22.87618
01/01/2016 02:17;1;-43.29333;-22.90662
01/01/2016 02:28;0;-43.54172;-23.01242
01/01/2016 02:44;2;-43.35943;-23.01030
01/01/2016 02:51;0;-43.35715;-22.85183
01/01/2016 02:56;3;-43.63401;-23.00212
01/01/2016 02:57;2;-43.25316;-22.99458
01/01/2016 03:00;0;-43.39274;-22.82499
01/01/2016 03:01;1;-43.36692;-22.86628
01/01/2016 03:03;2;-43.51445;-22.87363
01/01/2016 03:05;2;-43.60198;-22.91049
01/01/2016 03:06;3;-43.17507;-22.96576
01/01/2016 03:07;3;-43.29519;-22.83136
01/01/2016 03:11;3;-43.68198;-22.97946
01/01/2016 03:18;3;-43.18982;-22.96926
01/01/2016 03:24;1;-43.46151;-23.02613
01/01/2016 03:26;3;-43.18726;-22.97490
```

This file contains in the first column a date and time information (column **date_time** with date, hour, and minutes information), in the second column a priority (an integer between 0 and 3), in the third column **long** the longitudes, and in the fourth column **lat** the latitudes. To load this data with LASPATED, we first read the corresponding CSV file `events.csv` with

```
events = pandas.read_csv("events.csv").
```

Then the columns of the file are passed as specific arguments **datetime_col**, **lat_col** and **lon_col** of LASPATED method **add_events_data**. Additional information can be added inside a list passed to **feature_cols** argument. In the example above, the additional information is the priority of the call, provided in column **priority**. Therefore, the data from this file is loaded with LASPATED using

```
app.add_events_data(events, datetime_col='date_time', lat_col='lat',
                    lon_col="long", feature_cols=['priority'],
                    datetime_format='%d/%m/%Y %H:%M:%S').
```

Time-reference data can come in different formats, and the most common ones will be automatically detected and understood by LASPATED. A specific time column format using the **datetime_format** argument can be used, as shown in the example above. This format should be defined based on the datetime library pattern, see <https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior> for details.

We summarize in Example 1 the commands to load the events given in the file events.csv whose 19 first lines are shown above.

Listing 1: Building a data aggregator and adding events

```
1  import laspated as spated
2
3  # The data aggregator constructor receives a string with the coordinate ↵
   # reference system
4  app = spated.DataAggregator(crs="epsg:4326")
5
6  # Creating a pandas dataframe with events. It must have columns with the ↵
   # timestamp, latitude and longitude and it may have other features
7  events = pandas.read_csv("events.csv")
8
9  # The add_events_data method stores the events in the aggregator. The user ↵
   # must provide the names of columns for the timestamps, locations and the ↵
   # list of columns for the features of interest.
10 app.add_events_data(events, datetime_col='date_time', lat_col='lat', lon_col=↵
    "long", feature_cols=['priority'],datetime_format='%d/%m/%Y %H:%M:%S')
11
12 # The data aggregator stores the events added in the events_data dataframe
13 # Prints the first 19 observations, corresponding to the 19 observations ↵
   # shown above.
14 print(app.events_data.head(19))
```

4 Time discretization

4.1 Periodic with elementary time windows repetition

LASPATED provides multiple types of time discretization via **add_time_discretization** method. The simplest discretization is obtained using periodic time windows, where each period is divided into so-called elementary time windows of the same duration. The duration of this elementary time window is specified in the second argument of method **add_time_discretization** while the time unit of this duration is given in the first argument, which is a string with possible values 'Y' for years, 'M' for months, 'W' for weeks, 'D' for day, 'H' for hours, 'm' for minutes, and 'S' for seconds. The duration of a period, in the time unit of the first argument of the method, is specified in the third argument of the method. Therefore, the third argument needs to be a multiple of the second argument: it is the duration of an elementary time window times the number of elementary time windows in a period. In Example 2, we consider two such discretizations. In the first one, we want to count the number of arrivals for every day of the week. Therefore, the first argument of **add_time_discretization** is 'D' (the time unit is a day), the second argument is 1 (an elementary time window lasts 1 day), and the third argument is 7 (the one-week period has 7 days). In the second discretization, we want to count the number of arrivals in time windows of 30 minutes every day. Therefore, the first argument of **add_time_discretization** is 'm' (the time unit is a minute), the second argument is 30 (an elementary time window lasts 30 minutes), and the third argument is 30*48 (the one-day period has 30*48 minutes).

Listing 2: Elementary time windows repetition.

```

1 # 7 days in a week
2 app.add_time_discretization('D', 1, 7)
3
4 # 48 30-minutes slots in a day
5 app.add_time_discretization('m', 30, 30*48)

```

4.2 Periodic with elementary time windows of different durations

We can modify the previous time discretization keeping periodic observations but with elementary time windows of different durations. The same method `add_time_discretization` will be used with the first argument still providing the time unit ('Y', 'M', 'W', 'D', 'H', 'm', or 'S'). The second argument now is a list providing the durations of successive elementary time windows. For instance, in Example 3 which follows, the time unit is 'M' (month) and the second argument is the list [3, 4, 2, 1, 2] providing successive elementary time windows of 3 months, 4 months, 2 months, 1 month, and 2 months. The third argument is the period, which should be a multiple of the sum of the durations specified in the list of the second argument. In Example 3 the period is 12 months meaning that we count the number of observations for January-February-March in a first group, for April-May-June-July in a second group, for August-September in a third group, for October in a fourth group, and for November-December in a fifth group. In this example, if the last argument was 24 instead of 12 we would have 10 groups of time observations: observations for January-February-March of years 1, 3, 5,..., (with the years counted from the first year of historical data) in a first group, for April-May-June-July of years 1, 3, 5,..., in a second group, for August-September of years 1, 3, 5,..., in a third group, for October of years 1, 3, 5,..., in a fourth group, for November-December of years 1, 3, 5,..., in a fifth group, for January-February-March of years 2, 4, 6,..., in a sixth group, for April-May-June-July of years 2, 4, 6,..., in a seventh group, for August-September of years 2, 4, 6,..., in a eighth group, for October of years 2, 4, 6,..., in a ninth group, and for November-December of years 2, 4, 6,..., in a tenth group.

Listing 3: Elementary time windows of different durations.

```

1 # this creates an index that repeats yearly (12 months)
2 # January, February, March => observations indexed by time index 0
3 # April, May, June, July => observations indexed by time index 1
4 # August, September => observations indexed by time index 2
5 # October => observations indexed by time index 3
6 # November, December => observations indexed by time index 4
7 app.add_time_discretization('M', [3,4,2,1,2], 12)

```

4.3 Customized time intervals

We can specify more general time windows like holidays or special events defined by the user. LASPATED allows such flexibility by defining a set of customized time indexes for time intervals. The user needs to provide a DataFrame with the following columns:

1. start: index start time. When the period for this index starts.
2. end: index end time. When the period for this index ends.
3. t: index number. The index of the corresponding time period will receive.
4. repetition: informs whether the index should be repeated in different years ("yearly").

The customized discretization defined by these parameters will be applied to the data. In Example 4, for data given for years 2016 and 2017, we define a customized discretization that considers additional time windows for holiday January 1 (this holiday is yearly, and the corresponding observations will receive index 1), and for Carnival. For Carnival, the observations will receive index 2 and the **repetition** column is None, since for every year the specific dates of Carnival are given (02-06 to 02-11 for 2016 and 02-24 to 03-06 for 2017). Arrivals that do not belong to a customized time interval receive the index zero. Also note that a yearly defined customized time interval must begin and end the same year.

Listing 4: Customized time index.

```

1 import pandas as pd
2
3 time_disc_df = pd.DataFrame([
4     ["2016-01-01", "2016-01-01", 1, "yearly"],
5     ["2016-02-06", "2016-02-11", 2, None],
6     ["2017-02-24", "2017-03-06", 2, None],
7 ], columns=["start", "end", "t", "repetition"])
8
9 app.add_time_discretization(time_disc_df)

```

5 Space discretization

5.1 Defining borders

The first step for space discretization is to define the border of a region that contains the arrivals. The border is the region that will next be discretized in space. This can be done with LASPATED either using a custom map or a region automatically built from data (a rectangular region or an approximate convex hull of arrivals).

5.1.1 Custom map

LASPATED method `add_max_borders` adds a `max_border` attribute to object `app`. The parameter `data` of this method is a `GeoDataFrame` object². This `GeoDataFrame` object must have an attribute "geometry" encoding the vertices of the border and this object is obtained by calling the function `gpd.read_file` which takes as argument a Shapefile (with `.shp` extension) of the custom map (region under consideration). In Example 5 below, we show how to use `add_max_borders` on two examples, with data lying in Rio de Janeiro for the first example and data in New-York for the second. The respective shape files `rj.shp` and `ny.shp` can be found on the internet and also in folders `Data/rj` and `Data/ny` of the GitHub project.

Listing 5: Adding borders from a custom map.

```

1 import geopandas as gpd
2 import matplotlib as plt
3
4 # GeoDataFrame object read from Rio de janeiro shape file
5 custom_map = gpd.read_file(r'../Data/rj/rj.shp')
6 # adding as border
7 app.add_max_borders(data=custom_map)
8
9 # Plotting the border
10 app.max_borders.plot()
11 plt.show()
12
13 # GeoDataFrame object read from New-York shape file
14 custom_map = gpd.read_file(r'../Data/ny/ny.shp')
15 # adding as border
16 app.add_max_borders(data=custom_map)
17
18 # Plotting the border
19 app.max_borders.plot()
20 plt.show()

```

²see <https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.html>

5.1.2 Rectangular border and convex hull

If a customized border is not available, we can use a rectangular border containing all events or an approximate convex hull of the space observations. In the first case, the argument of parameter `method` of LASPATED method `add_max_borders` is "rectangle" while in the second case this parameter `method` is "convex". The following Example 6 shows how to define these borders. Only one of these two options must be chosen.

Listing 6: Automatically defined borders.

```
1 # adding rectangular borders
2 app.add_max_borders(method="rectangle")
3
4 # adding as border the convex hull of observations
5 app.add_max_borders(method="convex")
6
7 # plotting borders and a sample of the events
8 app.plot_discretization()
```

Once borders are defined, the space discretization step will split the corresponding region into subregions (depending on the discretization scheme) and link these regions to the arrivals. This step is performed with LASPATED method `add_geo_discretization`.

5.2 Rectangular discretization

A rectangular discretization of the studied area defined in the previous section is obtained using LASPATED method `add_geo_discretization` and specifying the value 'R' for parameter `discr_type`, and providing parameters `rect_discr_param_x` and `rect_discr_param_y` which respectively define the horizontal and vertical discretization steps. For instance, to generate a discretization into $10 \times 30 = 300$ rectangles, we use the parameters' `rect_discr_param_x=10` and `rect_discr_param_y=30`. The code of two rectangular discretizations for the city of Rio de Janeiro (using the borders of the city given by the corresponding shape file) is given in Example 7: the first discretization is a 10×10 discretization in 100 rectangles (resulting in the rectangular discretization given in Figure 1) while the second discretization is a 100×100 discretization in 10,000 rectangles (resulting in the rectangular discretization given in Figure 2).

Listing 7: Rectangular discretization.

```
1 # Discretization in 10x10=100 rectangles
2 # The corresponding discretization for Rio de Janeiro city is given in Figure 1
3 app.add_geo_discretization(
4     discr_type='R',
5     rect_discr_param_x=10,
6     rect_discr_param_y=10
7 )
8 # Plotting the regions
9 import matplotlib as plt
10 app.geo_discretization.boundary.plot()
11 plt.show()
12
13 # Discretization in 100x100=10 000 rectangles
14 # The corresponding discretization for Rio de Janeiro city is given in Figure 2
15 app.add_geo_discretization(
16     discr_type='R',
17     rect_discr_param_x=100,
18     rect_discr_param_y=100
19 )
20
21 # Plotting the regions
22 app.geo_discretization.boundary.plot()
23 plt.show()
```

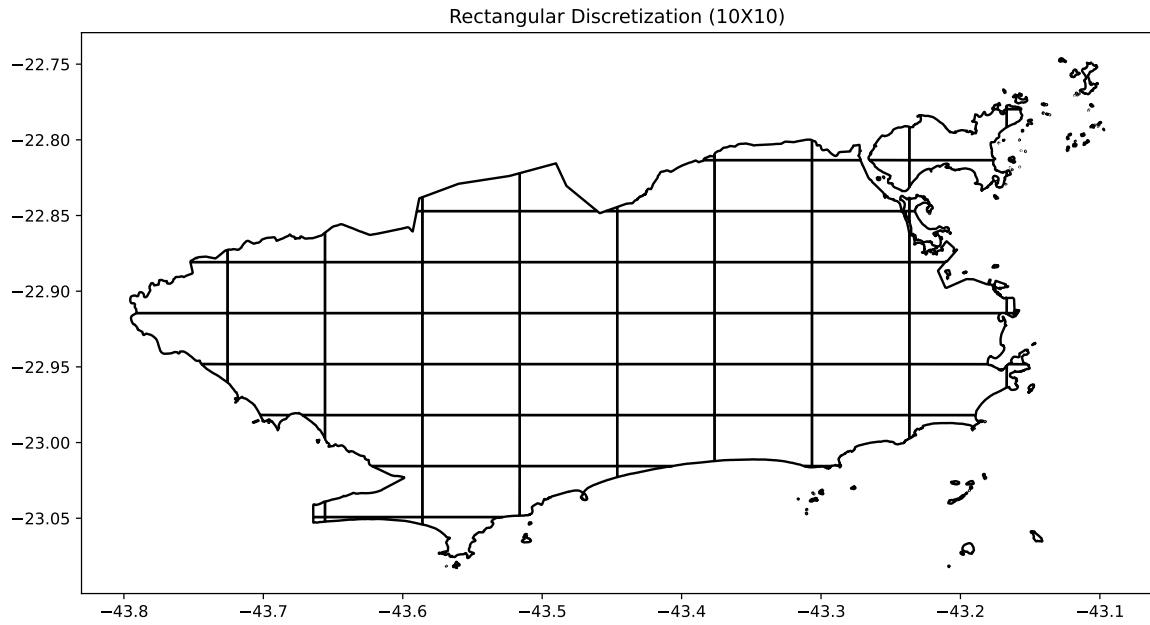


Figure 1: Space discretization of a rectangle containing the city of Rio de Janeiro in $10 \times 10 = 100$ rectangles, 76 of which have nonempty intersection with the city and are represented in the figure.

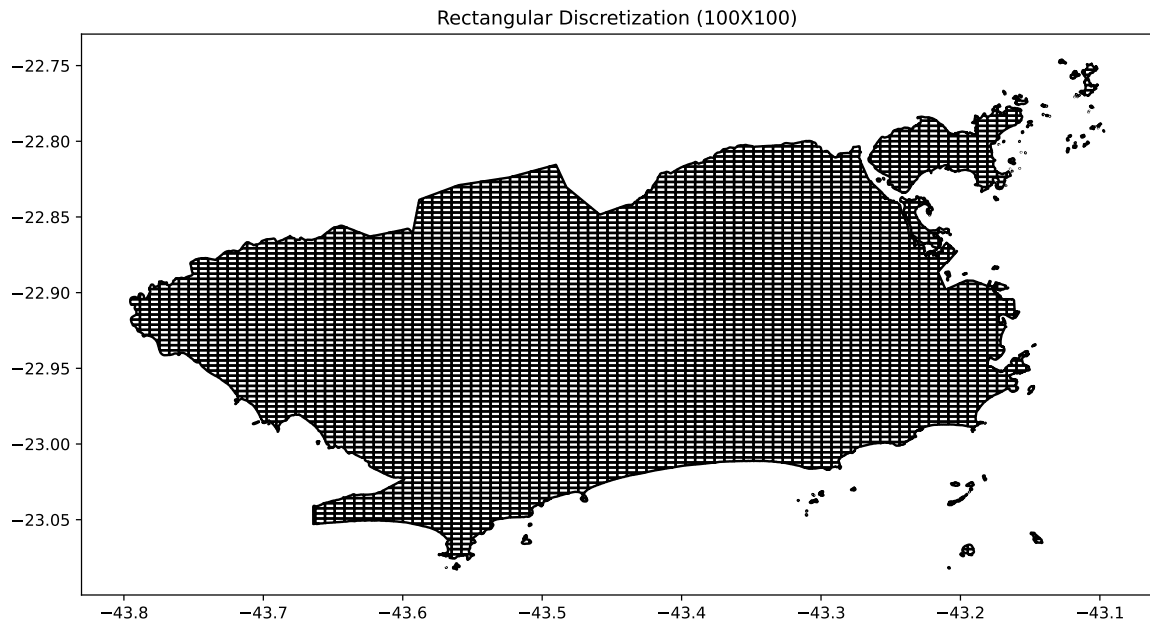


Figure 2: Space discretization of a rectangle containing the city of Rio de Janeiro in $100 \times 100 = 10\,000$ rectangles, 4916 of which have nonempty intersection with the city and are represented in the figure.

5.3 Hexagonal discretization

LASPATED also provides a hexagonal discretization of the studied area (we have used Uber Python package *H3* to generate the discretization).

This discretization is again obtained with LASPATED method **add_geo_discretization** now specifying 'H' for parameter **discr_type** and an additional integer parameter **hex_discr_param** taking values between 1 and 16, where the smaller the integer the coarser the discretization. Figures 3 and 4 show two hexagonal discretizations for the city of Rio de Janeiro obtained with Example 8 below: the first discretization uses scale parameter **hex_discr_param** equal to 7 while the second sets its scale to 8.

Listing 8: Hexagonal discretization.

```
1
2 # Hexagonal discretization with scale parameter 7
3 # The corresponding discretization for Rio de Janeiro city is given in Figure 3
4 app.add_geo_discretization(
5     discr_type='H',
6     hex_discr_param=7
7 )
8
9 import matplotlib as plt
10 app.geo_discretization.boundary.plot()
11 plt.show()
12
13 # Hexagonal discretization with scale parameter 8
14 # The corresponding discretization for Rio de Janeiro city is given in Figure 4
15 app.add_geo_discretization(
16     discr_type='H',
17     hex_discr_param=8
18 )
19
20 # Plotting the regions
21 app.geo_discretization.boundary.plot()
22 plt.show()
```

5.4 Customized discretization

LASPATED also allows discretization with customized subregions. First, a GeoDataFrame object containing the geometries of each subregion needs to be provided. Then the space discretization is obtained again with method **add_geo_discretization** specifying 'C' as **discr_type** parameter and the GeoDataFrame object of the subregions as the additional parameter **custom_data**. For instance, in Example 9 where the studied area is the city of Rio de Janeiro, we take as subregions the administrative districts. The corresponding GeoDataFrame object, denoted by **custom_map** in the example, is obtained from the Shapefile **rio_neighborhoods.shp** of the districts of the city (this shape file can be found on the GitHub page of the project). Figure 5 displays the corresponding customized discretization by administrative districts.

Listing 9: Custom discretization.

```
1 import geopandas as gpd
2 custom_map = gpd.read_file(r'../Data/rio_de_janeiro_neighborhoods/↵
    rio_neighborhoods.shp')
3 app.add_geo_discretization('C', custom_data=custom_map)
4
5 # Plotting the regions
6 app.geo_discretization.boundary.plot()
7 plt.show()
```

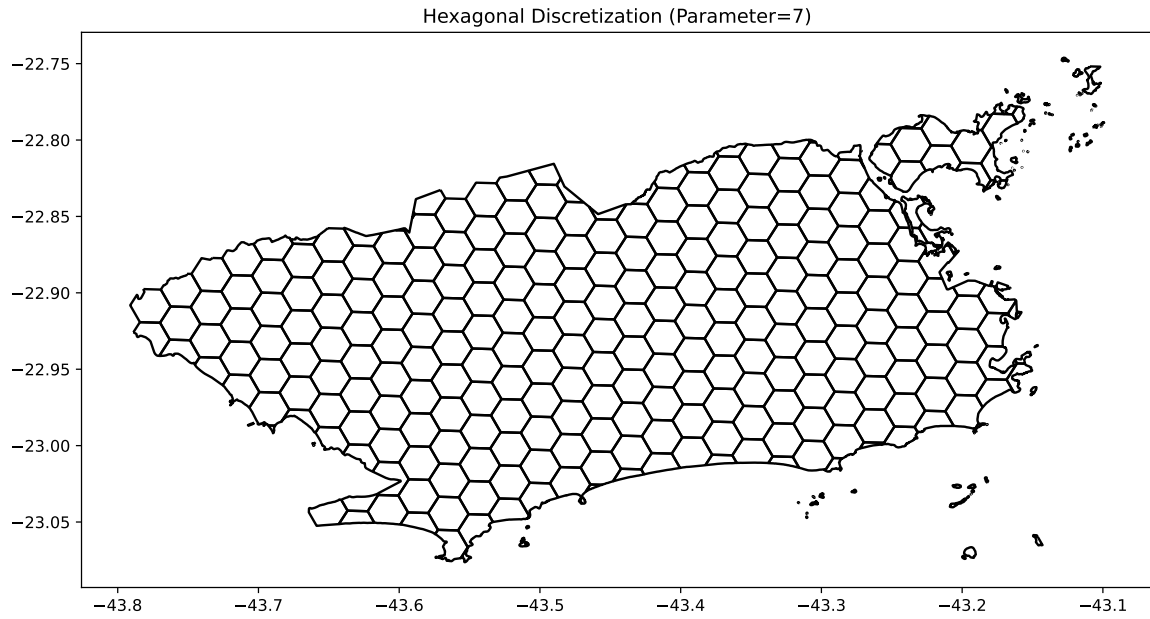



Figure 3: Space discretization of a rectangle containing the city of Rio de Janeiro in hexagons using Uber library H3 for space discretization in hexagons with scale parameter equal to 7.

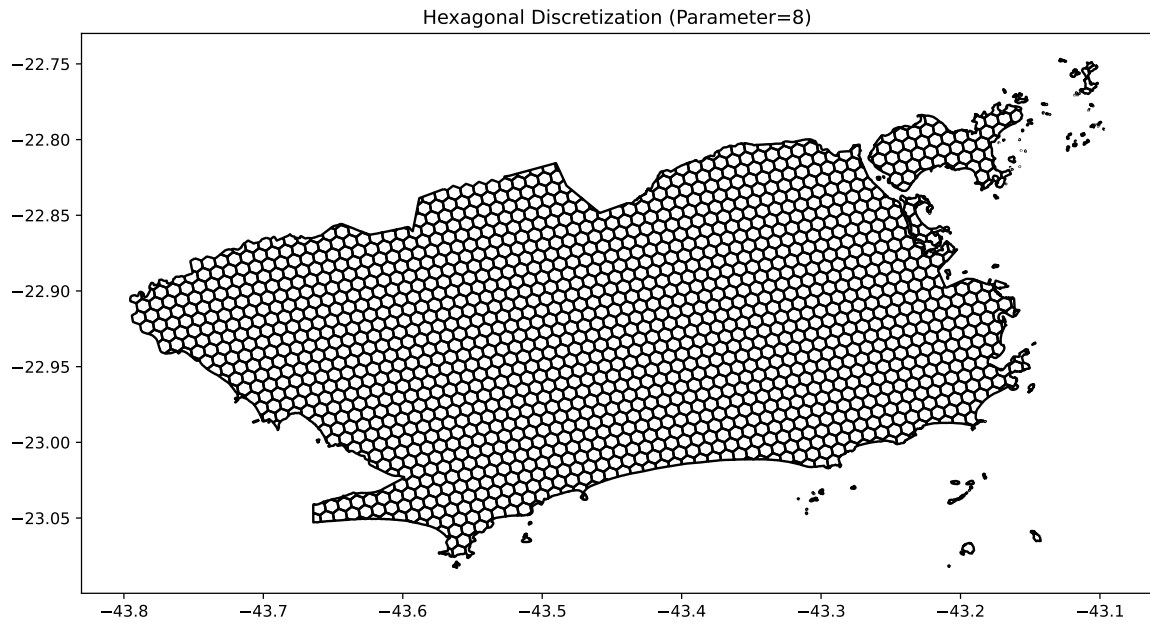


Figure 4: Space discretization of a rectangle containing the city of Rio de Janeiro in hexagons using Uber library H3 for space discretization in hexagons with scale parameter equal to 8.

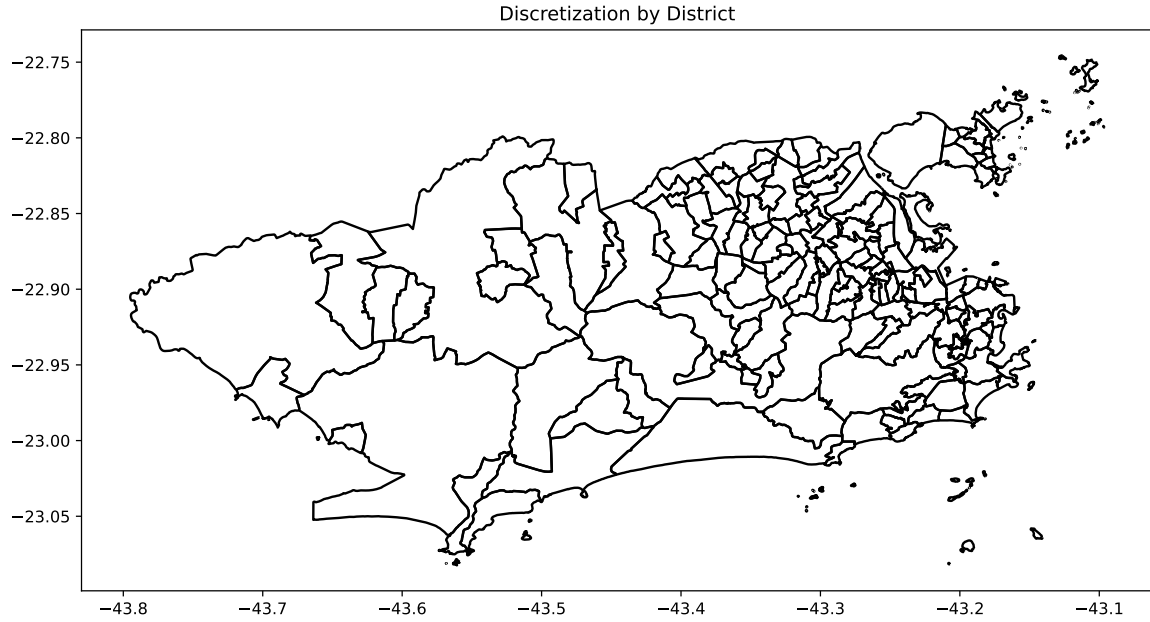


Figure 5: Space discretization of the city of Rio de Janeiro into 160 administrative districts.

5.5 Voronoi discretization

Given a set of locations $L = \{\ell_1, \ell_2, \dots, \ell_n\}$, LASPATED allows a discretization based on the Voronoi diagram given by the vertices in L . In our context, a Voronoi diagram is a partition of the border, given by the distances to each location in L . In the diagram, zone r_i contains all locations that are the closest to location ℓ_i than any other location in L . An application of Voronoi diagrams is to define possible covering regions for facilities (hospitals, schools, etc.).

Example 10 loads a GeoDataFrame object corresponding to the ambulance bases in Rio de Janeiro (we read the corresponding Shapefile bases.shp). Then it obtains the Voronoi discretization using again Laspatated method `add_geo_discretization` by setting 'V' as **discr_type** parameter and providing the bases as a GeoDataFrame object in parameter **custom_data**. Figure 6 displays the corresponding discretization given by the Voronoi diagram, with the dots corresponding to ambulance base locations.

Listing 10: Voronoi discretization.

```

1 import geopandas as gpd
2 bases = gpd.read_file("bases/bases.shp")
3 app.add_geo_discretization('V', custom_data=bases)
4
5 # Plotting the regions
6 app.geo_discretization.boundary.plot()
7 plt.show()

```

6 Additional discretization features

In the previous section, we have seen how to define a spatial discretization, say discretization D_1 . Let $\mathcal{I}$ be the index of the regions of this spatial discretization. Assume we have another discretization of the studied area, say discretization D_2 , with regions indexed by $\mathcal{J}$. In this context, LASPATED provides the following two functionalities:

- **get_intersection(gpd1, gpd2)**: it computes the areas $\mathcal{A}(i, j)$ of the intersections of regions $i \in \mathcal{I}$ and $j \in \mathcal{J}$. Parameters `gpd1` and `gpd2` are the GeoDataFrames corresponding to discretizations D_1 and D_2 .

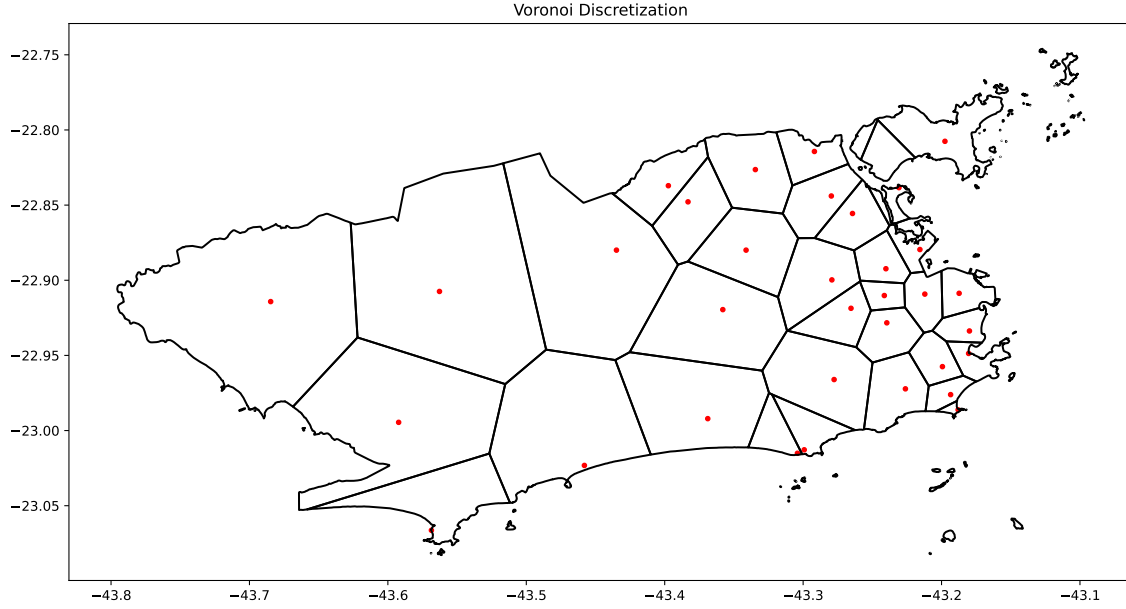


Figure 6: Discretization into 34 regions of the city of Rio de Janeiro given by the Voronoi diagram of ambulance bases in Rio de Janeiro.

D_2 . The method returns the areas $\mathcal{A}(i, j)$ as a 2-D NumPy array, with the areas measured in square kilometers.

- **app.add_geo_variable(gpd2, type_geo_variable).** Consider a given attribute, for instance the population and assume that for every region $j \in \mathcal{J}$ of discretization D_2 , we have a density d_j of this attribute; the attribute being uniformly distributed in the region. Let $P_j = d_j \mathcal{A}(j)$ be the value of the attribute in region $j \in \mathcal{J}$ where $\mathcal{A}(j)$ is the area of region j . When **type_geo_variable** is equal to "feature", LASPATED computes the value of the attribute $\sum_{j \in \mathcal{J}} d_j \mathcal{A}(i, j) = \sum_{j \in \mathcal{J}} \frac{P_j}{\mathcal{A}(j)} \mathcal{A}(i, j)$ for all regions $i \in \mathcal{I}$.

This calculation is simply obtained knowing attributes (P_j) and once areas $\mathcal{A}(j)$ and $\mathcal{A}(i, j)$ (using `get_intersection`) have been computed. The corresponding computation is done with LASPATED using method `add_geo_variable`. In Example 11, we consider the attribute population. We first read the corresponding shape file `population.shp` generating a `GeoDataFrame` object and filter only the columns "population" and "geometry". This file can be found on the project GitHub page and gives the population in all the districts of the city of Rio de Janeiro. Therefore, for this example, discretization D_2 is given by the districts of Rio de Janeiro (see Figure 5). The `GeoDataFrame` object filtered as explained above is passed as an argument of method `add_geo_variable`. The object `app` will contain information about the population of every elementary cell of discretization D_1 (see also the next section).

If we assume the index set $\mathcal{J}$ for the second discretization D_2 is partitioned into k index sets $\mathcal{J}_1, \dots, \mathcal{J}_k$: $\mathcal{J} = \sum_{i=1}^k \mathcal{J}_i$, then LASPATED method `add_geo_variable` can be invoked with **type_geo_variable** equal to "area" can also be used to compute $\mathcal{A}_{i\ell} = \sum_{j \in \mathcal{J}_\ell} \mathcal{A}(i, j)$ for every $\ell = 1, 2, \dots, k$, and $i \in \mathcal{I}$. In

Listing 11, we read a shape file `soil_use.shp` (available on the GitHub page of the project) providing the different land types (we have 16 of them in this data) for the city of Rio de Janeiro. Each elementary cell of this discretization contains one land type only. We then filter the first two land types. It then suffices to pass the corresponding `GeoDataFrame` object to method `add_geo_variable`. The object `app` will contain information about the areas of land types 1 and 2 for every elementary cell of discretization D_1 (see the next section).

Listing 11: Additional discretization features.

```

1 import geopandas as gpd
2 population = gpd.read_file(r'../Data/regressors/population/population.shp')
3 population = population[["population", "geometry"]].copy()
4 app.add_geo_variable(population, type_geo_variable='feature')
5
6 land_use = gpd.read_file(r'../Data/regressors/soil_use/soil_use.shp')
7 land_use = land_use[["type_1", "type_2", "geometry"]].copy()
8 app.add_geo_variable(land_use, type_geo_variable='area')

```

7 Retrieving and writing discretization data in files

LASPATED spatio-temporal discretizations are stored in the dataframe **app.events_data** and in the GeoDataFrame **app.geo_discretization**. Dataframe **app.events_data** has the same columns as the events dataframe loaded in Section 2, plus time and spatial discretization columns. For each event, the time index discretizations are added with the columns named **tdiscr_i**, $i = 0, \dots, T - 1$, with T being the number of time index discretizations defined. Custom column names may be provided in the **app.add_time_discretization** method, with the default **column_name** parameter. Similarly, the spatial discretization is added with the default name **gdiscr**. For each event, **gdiscr** is the index of the region where the event took place. The regions defined in the spatial discretization are fully detailed in the **app.geo_discretization** dataframe. For each region, the dataframe stores its index, the list of indexes of neighboring regions, the latitude/longitude of the centroid of the region, and columns with its additional discretization features (for instance population, areas of different land types, see Section 6 and Example 11 for these features).

LASPATED method **get_events_aggregated()** returns a NumPy multidimensional array of lists A , where $A[t_0, t_1, \dots, t_{T-1}, g, f_0, f_1, \dots, f_{F-1}]$ is a list containing the number of events observed in **app.events_data** for time discretizations $t_0, \dots, t_{T-1}$, spatial discretization g and features $f_0, \dots, f_{F-1}$ with T being the number of time discretizations and F the number of features. Example 12 shows how to call the method.

Listing 12: Aggregating events by discretization.

```

1 app.add_events_data(events, datetime_col='date_time', lat_col='lat', lon_col="↵
  long", feature_cols=['priority'])
2
3 app.add_time_discretization('m', 30, 30*48) # tdiscr_0
4 app.add_time_discretization('D', 1, 7) # tdiscr_1
5
6 app.add_geo_discretization(
7     discr_type='R',
8     rect_discr_param_x=10,
9     rect_discr_param_y=10
10 )
11
12 events_agg = app.get_events_aggregated()
13
14 # Axis are respectively: tdiscr_0, tdiscr_1, gdiscr, priority
15 print(events_agg.shape)

```

The method **write_arrivals(path_arrivals)** calls the **get_events_aggregated()** method and saves A in the path given by **path_arrivals**. The method **write_regions(path_regions)** saves information about the space discretization in the path given by **path_regions**, see Example 13 and the corresponding files on the github page of the project.

Listing 13: Writing the discretized data to a file.

```

1 app.write_arrivals("arrivals.dat")
2 app.write_regions("neighbours.dat")

```

8 Model calibration

8.1 C++ LASPATED calibration functions

LASPATED provides an implementation in C++ of the calibration functions. All code is contained in the header file `laspat.h`. The user can run the calibration functions by building a model object and a struct with parameters for the projected gradient and cross-validation functions. In the following sections, we explain how the C++ classes and methods work and show examples of the main functionalities.

8.1.1 Compilation and dependencies

C++ calibration functions use the following libraries:

- Gurobi (<https://www.gurobi.com>), an optimization toolbox used only in the model with covariates;
- `xtensor` and `xtl` libraries.

When compiling the code, the user needs to pass the location of the header files for `laspat`, `xtl`, `xtensor` and `gurobi`, as well as the location for the Gurobi libraries and their respective flags. `xtl` and `xtensor` require the C++14 standard and its flag must also be passed. The example below compiles an executable called `laspat` from a `main.cpp` that includes `laspat`. The example considers that the Gurobi solver will be used.

```
$ g++ -o laspat main.cpp -std=c++14 -I<gurobi_header_dir> -I<laspat_header_dir>
-I<xtl_header_dir> -I<xtensor_header_dir> -L<gurobi_libs_dir>
-lgurobi_c++ -lgurobi100
```

If Gurobi cannot be acquired, calibration functions for the model without covariates can still be used by passing the flag **-DUSE_GUROBI** equal to zero. The following command can be used to compile:

```
$ g++ -o laspat main.cpp -std=c++14 -DUSE_GUROBI=0 -I<laspat_header_dir>
-I<xtl_header_dir> -I<xtensor_header_dir>
```

We provide five test models in the source files `test_problems.cpp` and `test_problems.h` in the `Model_Calibration` directory. They were used to assert the quality of the projected gradient algorithm.

8.1.2 Classes and attributes

The C++ calibration implements a class **RegularizedModel** that represents the model without covariates and class **CovariatesModel** that represents the model with covariates. There is also a **Param** struct, containing all parameters for the projected gradient and cross validation functions. The **Param** struct has the following members:

- `double EPS`: tolerance used in the projection algorithm;
- `double sigma`: parameter used in the line search of the projected gradient method;
- `double accuracy`: gap used as stopping criterion for the projected gradient method;
- `int max_iter`: maximum number of iterations for the projected gradient method;
- `double lower_lambda`: lower bound for decision variables in the projected gradient method;
- `double upper_lambda`: upper bound for decision variables in the projected gradient method;
- `double beta_bar`: initial step size for the projected gradient method;
- `double cv_proportion`: proportion of samples used in each sub-iteration of the cross validation;
- `relax_empirical_fix`: if set to true, relaxes the constraint that makes the sum of estimated rates per class equal to the empirical rate by class.

Class **RegularizedModel** has the following attributes:

- param: reference for some parameter object;
- nb_observations: number of observations for each type-zone-time index;
- nb_arrivals: number of events for each type-zone-time index;
- durations: duration, in hours, for each time index;
- groups: description of time groups. groups[i][j] is the j-th time index in the i-th group;
- weights: time regularization weight for each time group;
- alpha: space regularization weight matrix of zones;
- distance: distance matrix (distance between zones);
- type_region: space groups. type_region[r] is the type of subregion r;
- neighbors: neighborhood description. neighbors[i][j] is the j-th neighbor of subregion i;
- which_group: groups of each time index. which_group[d][t] is the index of the time group corresponding to day d and time interval t within day d;
- C: number of event types;
- R: number of zones;
- T: number of periods.

The type of each attribute is defined according to the constructor method:

```
RegularizedModel(xt::xarray<int> &nb_observations,
                 xt::xarray<int> &nb_arrivals,
                 std::vector<double> &a_durations,
                 std::vector<std::vector<int>> &a_groups,
                 std::vector<double> &a_weights, xt::xarray<double> &a_alphas,
                 xt::xarray<double> &a_distance, std::vector<int> &a_type,
                 std::vector<std::vector<int>> &a_neighbors, Param &a_param).
```

The **CovariatesModel** has the following attributes:

- env: Gurobi environment object;
- param: reference for some parameter object;
- nb_observations: number of observations for each type-zone-time index;
- nb_arrivals: number of events for each type-zone-time index;
- durations: duration, in hours, for each time index (d, t).
- regressors: array of regressor values. regressor(j,r) is the value of the j-th regressor for zone r;
- C: number of event types;
- D: number of days;
- T: number of periods;
- R: number of zones;
- nb_regressors: number of regressors;

with corresponding constructor:

```
CovariatesModel(xt::xarray<int> &nb_observations,
                xt::xarray<int> &nb_arrivals,
                xt::xarray<double>& a_durations, xt::xarray<double> &reg,
                Param &param).
```

Both model classes have the same methods, with different implementations. The methods are used in the calibration functions and to assess the quality of its solutions:

- `double f(xt::xarray<double> &x)`: returns the value of the objective function at x ,
- `xt::xarray<double> gradient(xt::xarray<double> &x)`: returns the gradient of f at x ,
- `xt::xarray<double> projection(xt::xarray<double> &x)`: returns the projection of x onto the set defined by the constraints of the model with covariates,
- `bool is_feasible(xt::xarray<double> &x)`: returns true if x is in the feasible set defined by the constraints of the model with covariates and false otherwise,
- `double get_rhs(xt::xarray<double> &grad, xt::xarray<double> &dir)`: returns the value of the directional derivative given by gradient $grad$ and direction dir .
- `double get_lower_bound(xt::xarray<double> &x, xt::xarray<double> &grad)`: returns objective function lower bound given x and gradient $grad$.
- `double average_rate_difference(xt::xarray<double> &x1, xt::xarray<double> &x2)`: returns average absolute difference in rates between $x1$ and $x2$.

8.1.3 Calibration of the model without regressors

Given the objects described in the previous section, the function:

```
template <typename Model> xt::xarray<double>
    projected_gradient_armijo_feasible(Model &model, Param &param,
    xt::xarray<double> &x)
```

solves the model using the projected gradient method, with parameters given by the `param` object and with initial solution x . The function uses C++ templates, meaning that `Model` can be replaced with **RegularizedModel** or **CovariatesModel**. The function returns the best solution found in the projected gradient method.

Example 14 calls the calibration method for a model without covariates. For simplicity, we omitted the code that initializes all the model attributes. We begin including `laspated` header and using the `laspated` namespace. Next, we build a **RegularizedModel** object `m`, a **Param** object `param` and then run the projected gradient method, storing the result in the `xarray` `lambda`.

Listing 14: Calibration of the model without regressors in C++

```
1
2 #include "laspated.h"
3 // All laspated functions in namespace laspated
4 using namespace laspated;
5
6
7 int main(int argc, char* argv[]){
8     // Load variables nb_observations, nb_arrivals, durations,
9     // groups, weights, alphas, distance, type_region, neighbors.
10    // Builds param object with default parameters.
11    Param param;
12    RegularizedModel m(nb_observations, nb_arrivals, durations, groups, weights, ↵
        alphas, distance, type_region, neighbors, param);
13
14    // Initial solution
```

```

15     xt::xarray<double> lambda0 = 0.1*xt::ones<double>(nb_observations.shape());
16     // Runs the solver
17     xt::xarray<double> lambda = projected_gradient_armijo_feasible<↵
        RegularizedModel>(m, param, lambda0);
18
19     return 0;
20 }

```

8.1.4 Cross Validation

Cross validation can also be run for the model without regressors. This function allows us to choose the penalization parameters of the model without regressors. The corresponding `cross_validation` function has the following signature:

```

CrossValidationResult cross_validation(Param &param, RegularizedModel &model,
                                     xt::xarray<int> &sample,
                                     std::vector<double> &group_weights)

```

where `CrossValidationResult` is the following struct:

```

typedef struct {
    double cpu_time;
    double weight;
    xt::xarray<double> lambda;
} CrossValidationResult;

```

Example 15 shows how to execute the cross validation. The user must set the parameter `cv_proportion` $\in (0, 1]$ in the **Param** object, build a **RegularizedModel** object and provide a vector of `cv_weights`. The cross validation will evaluate how the model performs considering each weight in `cv_weights` as both the space and time regularization penalties. The returned struct contains the time spent performing cross validation, the best weight and the solution of the model considering the best weight. Note that the alphas and weights passed when building `m` are ignored in the cross validation, being replaced with each weight in `cv_weights`.

Listing 15: Cross validation in C++

```

1  #include "laspat.h"
2  #include <vector>
3
4  using namespace laspat;
5  using namespace std;
6
7  int main(int argc, char* argv[]){
8      // After loading variables nb_observations, nb_arrivals,
9      // durations, groups, weights, alphas, distance,
10     // type_region, neighbors
11     // Builds param object with default parameters.
12     Param param;
13     // Sets proportion for cross validation
14     param.cv_proportion = 0.2;
15     RegularizedModel m(nb_observations, nb_arrivals, durations, groups, weights, ↵
        alphas, distance, type_region, neighbors, param);
16     vector<double> cv_weights{0.1,1,10};
17     CrossValidationResult result = cross_validation(param, m, cv_weights);
18
19     return 0;
20 }

```

8.1.5 Calibration of the model with regressors

Example 16 performs the calibration of a model with covariates. Function `projected_gradient_armijo_feasible` can still be used for projections. We pass both the class **CovariatesModel** and an instance of this class to the function. The `CovariatesModel` object must be built using the observations, arrivals and covariates (regressors) data. The `projected_gradient_armijo_feasible` returns the estimated β variables.

Listing 16: Calibration of the model with regressors in C++

```
1  #include "laspated.h"
2  // All laspated functions in namespace laspated
3  using namespace laspated;
4
5
6  int main(int argc, char* argv[]){
7      // Load variables nb_observations, nb_arrivals, and regressors.
8      // Builds param object with default parameters.
9      Param param;
10     //Builds model object
11     Covariates m(nb_observations, nb_arrivals, regressors);
12
13     // Initial solution
14     xt::xarray<double> beta0 = 0.1*xt::ones<double>(nb_observations.shape());
15     // Runs the solver
16     xt::xarray<double> beta = projected_gradient_armijo_feasible<CovariatesModel>(m, param, beta0);
17
18     return 0;
19 }
```

8.1.6 Laspaté C++ app

To those unfamiliar with C++, we also provide a `laspated` binary app that allows running the calibration functions. The user must provide a set of files describing the model and a set of configuration options, either via a configuration file or command line. The options describe the algorithmic parameters, model attributes, and file locations. The valid options are the following:

- double EPS: tolerance used in the projection algorithm;
- double sigma: parameter used in the line search of the projected gradient method;
- double accuracy: gap used as stopping criterion for the projected gradient method;
- int max_iter: maximum number of iterations for the projected gradient method;
- double lower_lambda: lower bound for decision variables in the projected gradient method;
- double upper_lambda: upper bound for decision variables in the projected gradient method;
- double beta_bar: initial step size for the projected gradient method;
- double cv_proportion: proportion of samples used in each iteration of cross validation;
- output_file: path where the `laspated` app will save the estimated intensities.
- string model_type: the type of model being solved, must be either "reg" (for the model with covariates) or "no_reg" (for the model without covariates);
- string method: the type of method being used. For the model with covariates, only "calibration" is supported (running the projected gradient algorithm once). For the model without covariates, can be either "calibration" or "cross_validation";

- string algorithm: the algorithm used. Can be either "feasible" for Armijo search along the feasible direction or "boundary" for Armijo search along the boundary. However "cross_validation" only supports the "feasible" option;
- string info_file: path of file containing general information about the model. See below for a precise description of the corresponding file.
- arrivals_file: path of file containing the samples of arrivals. See below for a precise description of the corresponding file.
- neighbors_file: path of file containing the description of each zone, including its type, its covariates, and its neighbors. See below for a precise description of the corresponding file.
- durations_file: path of file containing the durations for each time index. See below for a precise description of the corresponding file.
- alpha_regions_file: path of file containing the space penalizations $R \times R$ matrix. See below for a precise description of the corresponding file.
- time_groups_file: path of a file containing a description of time groups and the corresponding penalties in the regularized model. See below for a precise description of the corresponding file.
- double duration: array with the duration of each period.
- cv_weights_file: path of a file containing the weights to be used in cross validation. See below for a precise description of the corresponding file.
- relax_empirical_fix: flag that, when set, relaxes the constraint that makes the sum of estimated rates per class equal to the empirical rate by class.

Note that to use the model with covariates, the user only needs to specify the info, arrivals, neighbors, and duration files. Additionally, for the model without regressors, the user must specify the files alpha_regions_file and time_groups_file. When performing cross validation, the file cv_weights_file is also needed.

The file info_file must contain two lines. The first line must contain, separated by spaces, the number of periods T per day, the number of week days G , the number of zones R , the number of arrival types C , the number of covariates J and the number of holidays H for each year. The second line contains, for each day of the week and holiday, the number of observations N_g for that day or holiday. Values of index g from 0 to 6 are used for days of the week while holidays have indexes in $\{7, 8, \dots, 7 + H\}$.

The file arrivals_file contains the samples observed, where each line describes a sample. Each line contains, separated by spaces, the period $t \in [0, T)$, the day $g \in [0, G)$, the zone $r \in [0, R)$, the arrival type $c \in [0, C)$, the sample index $j \in [0, N_g)$, the number of arrivals observed, and a flag indicating whether the day is a holiday or not.

The file neighbors_file contains the description of each zone. Each line contains the zone index, two values representing the coordinates of the zone centroid, a number indicating the zone's land type, J values corresponding to the values of all J covariates, followed by a set of pairs of values (index,distance) indicating the distances of the considered zone to its neighbors (identified by their indexes).

The file alpha_regions_file describes the space penalization parameters. It contains a $R \times R$ matrix (with entries separated by spaces) where entry (i, j) corresponds to the weight in the objective function of the model without covariates if zones i and j are neighbors.

The file time_groups_file contains the description of time groups and their corresponding penalties. The first line must have the number G of time groups. The subsequent $D * T$ lines must have the group index of each period. Then the subsequent G lines may contain the weight of each group. If the method is set to "cross_validation", the reading of time weights is skipped.

File cv_weights_file contains the weights to be used in cross validation, separated by spaces. Each weight must be non-negative.

The durations file contains the duration of each time index, in hours. The first line contains, separated by spaces, the number of day indexes and the number of time indexes.

The LASPATED app is only supported on Linux operating systems. The user must have the GCC compiler and the Boost library. Libraries xtl and xtensor are also used and provided. In order to use the model with covariates, the user must have Gurobi installed. The directory Model_Calibration/Cpp contains a Makefile that is used to compile the app with:

```
$ make GUROBI_LIB_VER=-lgurobi110 GUROBI_HOME=/opt/gurobi1101/linux64
INCLUDE_BOOST=/usr/include/boost
```

The Makefile uses GUROBI_HOME to find Gurobi directory, GUROBI_LIB_VER to find the Gurobi C library version and INCLUDE_BOOST to find the Boost directory. The values displayed above are actually the default values set in the Makefile, so if Boost and Gurobi are installed in those locations, then the make command will compile the code. Once laspated app is successfully compiled, it can be run with:

```
$ ./laspated -f config
$ ./laspated -f --model_type=no_reg --method=calibration
--info_file=info.txt --arrivals_file=arrivals.txt
--neighbors_file=neighbors.txt
--alpha_regions_file=alpha_regions.txt
--time_groups_file=time_groups.txt
--durations_file=durations.txt
--output_file=output.txt
```

where config is the config text file with options. Note that, in the second example, the options provided in the command line override any options with the same name in the config file.

After running laspated app, the output is saved in the path provided by option output_file. If the model without covariates is used, the output contains on each line the indexes $c \in [0, C)$, $r \in [0, R)$, $t \in [0, D * T)$, and the intensity λ_{crt} estimated by LASPATED. If the model with covariates is used, then each line correspond to indexes $c \in [0, C)$, $d \in [0, D)$, $t \in [0, T)$, $j \in [0, J)$, and the parameter β_{cdtj} estimated by LASPATED.

9 Demo scripts

In the LASPATED source code repository, we provide a demo directory, containing examples for the discretization and calibration functions. The demo summarizes all code excerpts provided in the previous sections. The directory contains three Python files:

- demo_discretization.py: Contains examples of how to generate a discretization;
- demo_calibration_regularized.py: Contain an example of running the calibration and cross-validation using the regularized model (without covariates);
- demo_calibration_covariates.py: Contain an example of running the calibration model with covariates.

The discretization demo can be run with

```
$ python demo_discretization.py border_type discretization_type
```

where border_type can be: *rectangle*, *convex* or *custom* and discretization_type can be: *rectangle*, *hexagon* or *custom*. The border type defines the type of border: rectangular, convex or the border of Rio de Janeiro city (custom). The discretization type can be rectangular, hexagonal or according to Rio de Janeiro's administrative regions (custom). The script considers arrival data from Rio de Janeiro and produces output files considering three time discretizations, and the Rio de Janeiro population as an additional feature.

Before running the calibration demo, the user must compile the laspated executable, following instructions from section 8.1.6. The calibration demo for the model without covariates can be run with

```
$ python demo_calibration_regularized.py
```

Additionally, the calibration demo for the model with covariates can be run with:

```
$ python demo_calibration_covariates.py
```

The input files for the C++ functions and the output of the functions are saved inside the demo/calib_data respective subdirectories. The regularized model script runs the calibration function considering an unitary weight time and space regularization, and a cross-validation procedure considering a set of weights.

10 Replication script

Along with the LASPATED source code, we also provide a python script that performs all experiments from Guigues et al. (2023). Before running the script, the first step is to compile the C++ code. If you have a GCC compiler installed, this can be done by running the command:

```
$ cd LASPATED/Replication
$ make -C cpp_tests
```

The `cpp_tests` directory contains a Makefile that can be edited by the user that compiles the C++ script. The Makefile inside directory `cpp_tests` accesses the user-defined environment variable `$GUROBI_HOME`. If it is set, then the code for the model with Covariates inside `laspated.h` is accessible. Otherwise, the script will not run the experiments that use covariates.

After the C++ code is successfully compiled, you can run the replication script by simply running:

```
$ python replication_script.py
```

This command will generate the rectangular, hexagonal and district discretizations using the `laspated` python module, run the experiments with the C++ code and process the results, generating the figures and tables presented in the paper.

All results presented in the paper are saved in the directory `replication_results`, with subdirectories `plots` and `tables`. If you want access to the raw data generated by C++, check the directory `cpp_tests/results`, with subdirectories for each experiment made. Subdirectories `ex1` and `ex2` contain results for the first and second model without covariates (Examples 1 and 2). Subdirectory `ex3` contains results for the model with covariates (Table 2 of the paper) and `real_data` contains results for the experiments with real data of Rio de Janeiro. For more details, see Guigues et al. (2023).

11 Docker container

We also provide a Docker container. It is available at <https://dockerhub.com> or via the Dockerfile in the repository. The docker container comes with all dependencies installed and the library ready to use.

11.1 Dockerfile

To build the container via the repository Dockerfile, run:

```
docker build -t laspated .
```

If you have a Gurobi Web License, you can build the container with Gurobi support by running:

```
docker build --build-arg USE_GUROBI=1 -t laspated .
```

The above command will build the container with Gurobi 11.0.1 installed.

11.2 DockerHub

To download the container from DockerHub, just run:

```
docker pull victorvhrn/laspated
```

Note that the DockerHub container is built without Gurobi support by default. To enable Gurobi support, the C++ code must be recompiled inside the container. Check the next section on how to run the container and section 8.1.6 on how to recompile the C++ code.

11.3 Running the container

To run the container, use:

```
docker run -it laspated
```

To run the container with Gurobi support, you can pass a valid Gurobi Web license to the container with:

```
docker run --volume="/absolute/path/to/gurobi.lic:/opt/gurobi/gurobi.lic:ro"
-it laspated-dock
```

Both commands will open a shell environment with all dependencies installed. Once in the container environment, you can run both the Python and C++ functions.

12 Missing Data

An example of model estimation with missing data is available in subdirectory `Missing_Data`. The subdirectory contains the implementation of four models that consider missing location data, and one artificial example considering missing timestamp data.

The file `missing_data.hpp`, implements the regularized model (**class `MissingLambdaRegularizedModel`**) and a model that assumes the probability of a missing location for an arrival is proportional to the population in each zone (**class `MissingLambdaCovariatesModel`**). Additionally, `missing_data.hpp` implements a customized version of the cross validation procedure for the regularized model.

The file `missing_data.cpp` implements a function that performs the estimation using a closed form expression, a function that uses the Fisher information matrix to compute confidence intervals, a function that runs an artificial example with missing timestamps, and a function that runs a set of experiments on the models. This code corresponds to the tests performed in the numerical experiments section of Guigues et al. (2024).

Because of the Fisher information matrix computation, the missing data code requires the `xtensor-blas` and the `openblas` libraries. To compile the code, run:

```
cd Missing_Data
g++ -o missing Cpp/missing_data.cpp -DUSE_GUROBI=1 -std=c++14 -m64
-I../Model_Calibration/Cpp -I$GUROBI_HOME/include -L$GUROBI_HOME/lib
-lboost_program_options -lopenblas -O3 -lgurobi110 -lgurobi_c++
```

The signature of the cross validation function for missing data is given by:

```
laspated::CrossValidationResult cross_validation(
    laspated::Param& param, MissingLambdaRegularizedModel& model,
    xt::xarray<int>& sample, xt::xarray<int>& sample_missing_arrivals,
    std::vector<double>& group_weights)
```

The following example shows how to run the C++ code for missing data:

```
./missing -f test.cfg --model=all --info_file=info.dat
--neighbors_file=neighbors.dat --arrivals_file=arrivals.dat
--missing_file=missing_arrivals.dat
```

This example runs all missing data models for the input files provided. In general, options can be provided in a configuration file, the `test.cfg` file provided with `-f`, or in the command line. Note that the command line options override the values in the configuration file. The following options are identical to the `LASPATED` application:

- `EPS`,
- `sigma`,
- `max_iter`,
- `lower_lambda`,

- beta_bar,
- info_file,
- arrivals_file,
- neighbors_file.

The remaining options are specific to the missing data application. The option missing_file specifies the path to the file containing missing arrivals' data. This file follows the same format as arrivals_file, but the application ignores the value of the zone index. Option mc_samples specifies the Monte Carlo sample size S , used in the **MissingLambdaCovariatesModel**. The option test_weights contains a list of weights to be used in the **MissingLambdaRegularizedModel**.

The option model defines the missing data model(s) that the application will run. It can be one of the following:

- all: runs all four missing data models;
- analytical: runs only the closed form models;
- regularized: runs only the **MissingLambdaRegularizedModel**, once for each of the test_weights set, and once performing a cross validation;
- population: runs only the **MissingLambdaCovariatesModel**, with the number of samples provided by mc_samples.

For the **MissingLambdaCovariatesModel**, there is also the function mean_var_u, that computes the mean and variance of the Monte Carlo samples for an observation indexed by $c \in \mathcal{C}, d \in \mathcal{D}, t \in \mathcal{T}, n \in \mathcal{N}_{c,d,t}$. Its signature is:

```
std::pair<double, double> mean_var_u(xt::xarray<int>& mn_samples, int c,
                                     int d, int t, int n,
                                     xt::xarray<double>& x)
```

where mn_samples is the $S \times |\mathcal{I}|$ array of samples, (c, d, t, n) is the observation index and $x = (x_{c,d,t,i} : c \in \mathcal{C}, d \in \mathcal{D}, t \in \mathcal{T}, i \in \mathcal{I})$ is the Poisson intensities array. The function returns a pair containing the mean and variance, respectively.

12.1 Model comparison with mean squared error

Users can evaluate the capacity of the models to generalize to unseen periods using the measure_models_accuracy function:

```
xt::xarray<double> measure_models_accuracy(
    AppParams& app_params, xt::xarray<int>& sample,
    xt::xarray<int>& sample_missing_arrivals, xt::xarray<double>& durations,
    std::vector<std::vector<int>>& neighbors, std::vector<double>& pop,
    xt::xarray<double>& regressors)
```

This function evaluates model accuracy over time by splitting the observation index into month-like blocks, computing empirical rates for each block, fitting models on the complementary observations, and comparing estimated versus empirical rates via squared errors. Its inputs provide runtime settings (app_params), observed arrivals with location (sample), missing-location arrivals (sample_missing_arrivals), durations of each time interval (durations), spatial structure (neighbors), proportion of population per zone (pop), and covariates (regressors); it returns an xt::xarray<double> containing month-indexed error outputs while also writing detailed per-model results to results/accuracy_results.txt.

Users can invoke measure_models_accuracy by running the missing-location pipeline with the model option set to "accuracy" in run_models_missing_location, after providing the required input files (info, arrivals, missing, and neighbors) and loading the derived tensors (sample, sample_missing_arrivals, durations, neighbors, pop, and regressors).

References

- V. Guigues, A. J. Kleywegt, G. Amorim, A. M. Krauss, and V. H. Nascimento. LASPATED: A Library for the Analysis of SPAtio-TEmporal Discrete Data. arXiv:2401.04156v2 [stat.ME], 2023.
- Vincent Guigues, Anton Kleywegt, Victor Hugo Nascimento, and Lucas Lucas Rafael de Andrade. Estimation of spatiotemporal poisson processes with some missing location data, 2024. URL <https://arxiv.org/abs/2410.11103>.